



Università degli Studi dell'Insubria

Dipartimento di Scienze Teoriche e Applicate

Corso di laurea triennale in Informatica

Tesi di Laurea

**Implementazione di una rete basata
su microservizi tramite la piattaforma
Kubernetes**

Relatore

Ch.ma Prof.ssa Sabrina Sicari

Co-relatore

Prof.ssa Alessandra Rizzardi

Tesi di Laurea di

Federico Garegnani

Matricola 746789

Anno accademico

2022–2023

INDICE

1	Introduzione	3	
2	Stato dell'arte	5	
2.1	Architettura monolitica	5	
2.2	Deployment tradizionale	5	
2.3	Deployment su macchine virtuali	6	
2.4	Deployment su container	7	
3	Architettura del sistema	8	
4	Implementazione	9	
5	Risultati ottenuti	10	
6	Conclusioni e sviluppi futuri	11	
	Bibliografia	13	

INTRODUZIONE

Lo sviluppo di applicazioni distribuite rappresenta una disciplina fondamentale nell'evoluzione del mondo dell'informatica. Questa pratica si concentra sulla progettazione ed implementazione di sistemi software che operano su reti di computer interconnessi, consentendo la condivisione efficiente di risorse e dati. All'interno del panorama tecnologico odierno lo sviluppo di applicazioni distribuite risulta cruciale per soddisfare le esigenze di sistemi complessi.

In ambito di sviluppo delle applicazioni suddette si riconoscono due grandi modalità di approccio al problema: un approccio più tradizionale che vede il sistema da sviluppare come un'unica unità, ed un metodo più moderno che mira a frammentare il problema principale in unità di minori dimensioni cooperanti tra loro. Il primo approccio descritto segue un'architettura monolitica e prevede che l'intera applicazione sia sviluppata e distribuita come un unico componente; contrapposta a questa negli ultimi anni è emersa l'architettura a microservizi, un approccio architetturale che mira a scomporre la logica applicativa in componenti piccole ed autonome, consentendo lo sviluppo e il rilascio indipendenti dei singoli servizi[1].

L'approccio a microservizi porta con sé una serie di vantaggi:

SCALABILITÀ Possibilità di scalare singoli servizi in base alle esigenze senza influenzare l'intera applicazione.

INDIPENDENZA I diversi team possono lavorare in modo autonomo su diversi microservizi.

ADATTABILITÀ È possibile sfruttare tecnologie differenti per ciascun microservizio, adattandosi meglio a requisiti o compiti specifici.

MANUTENIBILITÀ Un problema ad un microservizio non si propaga all'intera applicazione ma rimane isolato all'interno di esso, facilitandone l'individuazione.

Obiettivo di questo progetto è sviluppare un semplice sistema gestionale di un supermercato secondo i due approcci sopra descritti e confrontarne infine le prestazioni.

L'applicazione prevede l'interazione con clienti che acquistano prodotti dal supermercato e per ogni acquisto implementa la gestione dell'ordine, dalla rimozione dei prodotti dal magazzino fino all'avvenuto pagamento. Possiamo vedere il servizio complessivamente offerto come l'e-commerce di un supermercato.

I clienti sono di due tipi:

- *Standard*. È interessato all'acquisto di prodotti e genera nuovi ordini;

- *Fornitore*. Si occupa di rifornire periodicamente il magazzino.

Gli ordini seguono questo percorso:

1. *Creato*. L'ordine è stato ricevuto dal sistema;
2. *Confermato*. È stato confermato che nel magazzino ogni prodotto è presente in quantità sufficiente per soddisfare l'ordine;
3. *Pagato*. Il pagamento dell'ordine è andato a buon fine;
4. *Spedito*. L'ordine è stato spedito ed è possibile verificare lo stato della spedizione.

Nei capitoli seguenti si descriveranno dapprima lo stile architetturale monolitico e a microservizi, li si confronterà e si elencheranno pro e contro, per poi concentrarsi nello specifico sull'applicazione sviluppata. In dettaglio:

- *Capitolo 2*: stato dell'arte nello sviluppo di applicazioni distribuite, relativi vantaggi e svantaggi di ogni approccio.
- *Capitolo 3*: architettura del sistema e tecnologie impiegate.
- *Capitolo 4*: spiegazione delle due soluzioni proposte.
- *Capitolo 5*: confronto di performance tra le due soluzioni.
- *Capitolo 6*: conclusioni e possibili sviluppi futuri.

STATO DELL'ARTE

In ambito di sviluppo e deployment di applicazioni distribuite si distinguono diversi metodi, ognuno con i propri peculiari vantaggi e svantaggi; descriveremo nel corso di questo capitolo pro e contro di ogni approccio.

2.1 ARCHITETTURA MONOLITICA

L'architettura monolitica è il modello tradizionale di sviluppo di un'applicazione in cui l'intera logica applicativa è implementata in un unico componente. Un simile approccio è ideale per applicazioni di dimensioni ridotte e che non devono soddisfare un elevato numero di richieste, presentano infatti una minore complessità di sviluppo ed offrono performance migliori[2]. All'aumentare della dimensioni dell'applicazione o del traffico da soddisfare il modello monolitico inizia però a mostrare i propri limiti:

- *Il sistema non è scalabile*: Se una singola parte dell'applicazione è soggetta a un carico di lavoro elevato, questa non può essere ridimensionata individualmente, ma si è obbligati a scalare l'intera applicazione.
- *Poca flessibilità*: Tutti i moduli devono obbligatoriamente utilizzare la stessa tecnologia, una modifica in questo senso comporterebbe la riscrittura di buona parte dell'applicazione. Alcuni vincoli potrebbero essere la scelta di un linguaggio di programmazione unico, di un framework specifico, di un unico sistema per la gestione del codice sorgente, di un'unica piattaforma di hosting.
- *Poca resilienza*: Un errore all'interno di un modulo può portare al blocco dell'intera applicazione.
- *Mantenimento*: Anche una singola modifica all'interno di un modulo costringe ad eseguire nuovamente il deploy dell'intera applicazione.

2.2 DEPLOYMENT TRADIZIONALE

La metodologia tradizionale di deployment prevede l'esecuzione delle applicazioni direttamente sul server fisico. Sebbene semplice ed immediato questo modo di procedere non è ottimale: non permettendo di scalare l'applicazione con dei limiti sull'utilizzo di risorse (CPU, RAM, disco, ...) si corre il rischio che una seconda applicazione non riesca a garantire le prestazioni desiderate. Si potrebbe cercare un'alternativa a questo



Figura 2.1: Confronto di una struttura monolitica e una a microservizi[2]

problema eseguendo ogni applicazione su un server fisico distinto, ma anche una tale soluzione non è adatta a un contesto reale risultando molto costosa e portando a un sottoutilizzo delle risorse complessive[1].

2.3 DEPLOYMENT SU MACCHINE VIRTUALI

Le macchine virtuali sono pacchetti software piuttosto pesanti in grado di eseguire l'emulazione dei dispositivi hardware di basso livello quali CPU, RAM, dischi, schede di rete. Il deployment su macchine virtuali (anche chiamato deployment virtualizzato) prevede l'esecuzione di più macchine virtuali sul server fisico, ciascuna delle quali esegue un sistema operativo che a sua volta esegue una o più applicazioni. Un simile approccio è in grado di far fronte ad alcune delle problematiche del deployment tradizionale quali:

- *Limitazione delle risorse:* In funzione di quanto detto sopra è possibile limitare le risorse disponibili per ad una macchina virtuale in termini di tempo di CPU, quantità di RAM, etc.
- *Isolamento:* Le macchine virtuali eseguono in un ambiente isolato e completamente autonomo che le rende immuni a tentativi di interferenza da parte di altre macchine virtuali in esecuzione sullo stesso host.

Dall'altro canto i principali svantaggi delle macchine virtuali riguardano la ridotta velocità di generazione e distribuzione delle stesse e il maggiore spazio di archiviazione richiesto: trattandosi di *immagini* di un sistema indipendente possono infatti richiedere diverso tempo per essere generate e configurate, l'affiancamento di diverse macchine

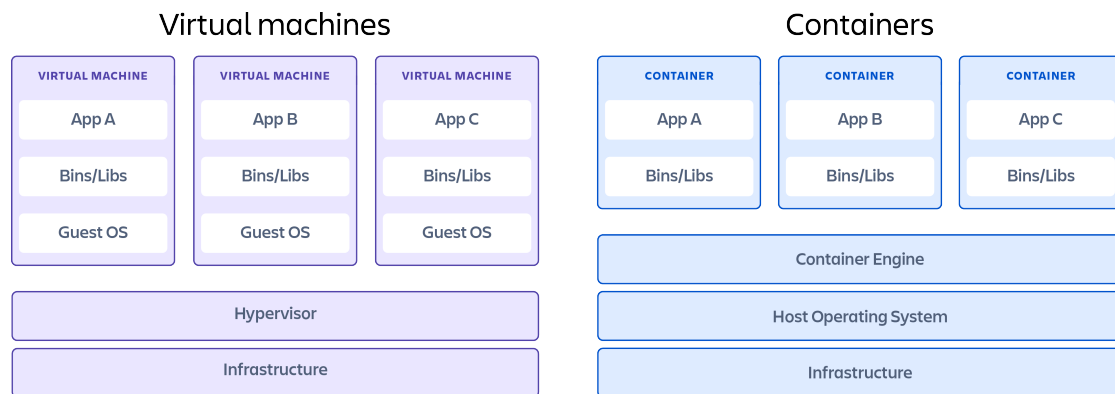


Figura 2.2: Struttura di un sistema basato su macchine virtuali e container [3]

virtuali porta inoltre alla duplicazione del sistema operativo installato su di esse, con un conseguente utilizzo di spazio di archiviazione.

2.4 DEPLOYMENT SU CONTAINER

Un *container* è un modello di virtualizzazione più leggero rispetto ad una macchina virtuale che racchiude un software e tutte le dipendenze necessarie ad eseguirlo. Al pari delle macchine virtuali i container rappresentano un ambiente isolato, con risorse limitate ed un proprio file system per l'esecuzione del software, ma non virtualizzano un intero sistema operativo. L'uso di container offre una grande efficienza in fase di sviluppo e di esecuzione, essi contengono solo software di alto livello e sono molto veloci da modificare, oltre al fatto che non dovendo virtualizzare un sistema operativo la loro esecuzione risulta molto più rapida e meno esosa di risorse hardware. Per finire molti sistemi runtime per container mettono a disposizione degli sviluppatori repository pubbliche con immagini preconfezionate di software popolari, il tutto al fine di avere uno sviluppo ancora più rapido [1, 3].

ARCHITETTURA DEL SISTEMA

IMPLEMENTAZIONE

RISULTATI OTTENUTI

CONCLUSIONI E SVILUPPI FUTURI

ELENCO DELLE FIGURE

Figura 2.2 Struttura di un sistema basato su macchine virtuali e container
[3] 7

BIBLIOGRAFIA

- [1] *Cos'è Kubernetes?* URL: <https://kubernetes.io/it/docs/concepts/overview/what-is-kubernetes/> (cit. alle pp. 3, 6, 7).
- [2] *Microservices vs. monolithic architecture – When monoliths grow too big it may be time to transition to microservices.* URL: <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith> (cit. a p. 5).
- [3] *Containers vs. virtual machines – Learn the differences between containers and virtual machines, popular providers for each, and how they can be used together.* URL: <https://www.atlassian.com/microservices/cloud-computing/containers-vs-vms> (cit. a p. 7).